

2026 г.

ВВЕДЕНИЕ

Целью выполнения лабораторных работ является освоение современных технологий веб-разработки, формирование практических навыков проектирования пользовательских интерфейсов и создание полноценных веб-приложений с применением актуальных инструментов.

В процессе работы реализован комплекс лабораторных заданий, направленных на поэтапное изучение:

- основ HTML и CSS с последующей реализацией веб-калькулятора в стилистике сайта визового центра РФ[1];
- принципов интерактивности с использованием JavaScript, включая динамическое обновление интерфейса;
- базовых возможностей платформы Node.js и npm, создание двухстраничного сайта с применением выбранного компонента в рамках заданной тематики;
- написания и интеграции алгоритмов различной сложности на JavaScript в общий проект;
- разработки и тестирования собственного API с поддержкой CRUD-операций и возможностью фильтрации данных.

ЛАБОРАТОРНАЯ РАБОТА №1 (Калькулятор HTML/CSS)

- **Задача:** Создание калькулятора. Верстка на HTML, CSS.

В рамках лабораторной работы был разработан калькулятор, содержащий:

- Поле для вывода результата.
- Кнопки операций и чисел.
- Кнопку перехода в github репозиторий.

- Используемые технологии

- HTML – для создания структуры страницы.
- CSS – для стилизации элементов.

- Выполненные задания

В процессе работы был разработан графический интерфейс калькулятора (Рисунок 1)

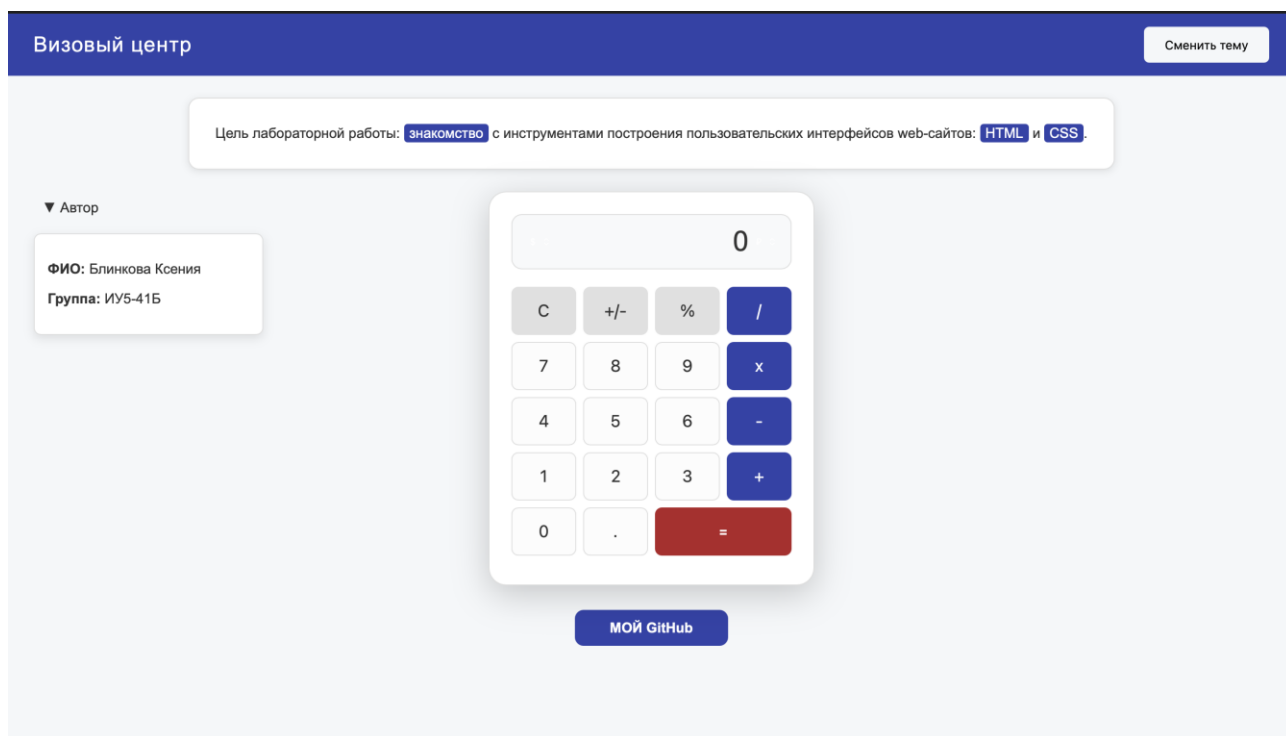


Рисунок 1 - Конечный вид страницы

Интерфейс оформлен по образцу сайта единого визового центра [1] (Рисунок 2, Рисунок 3).

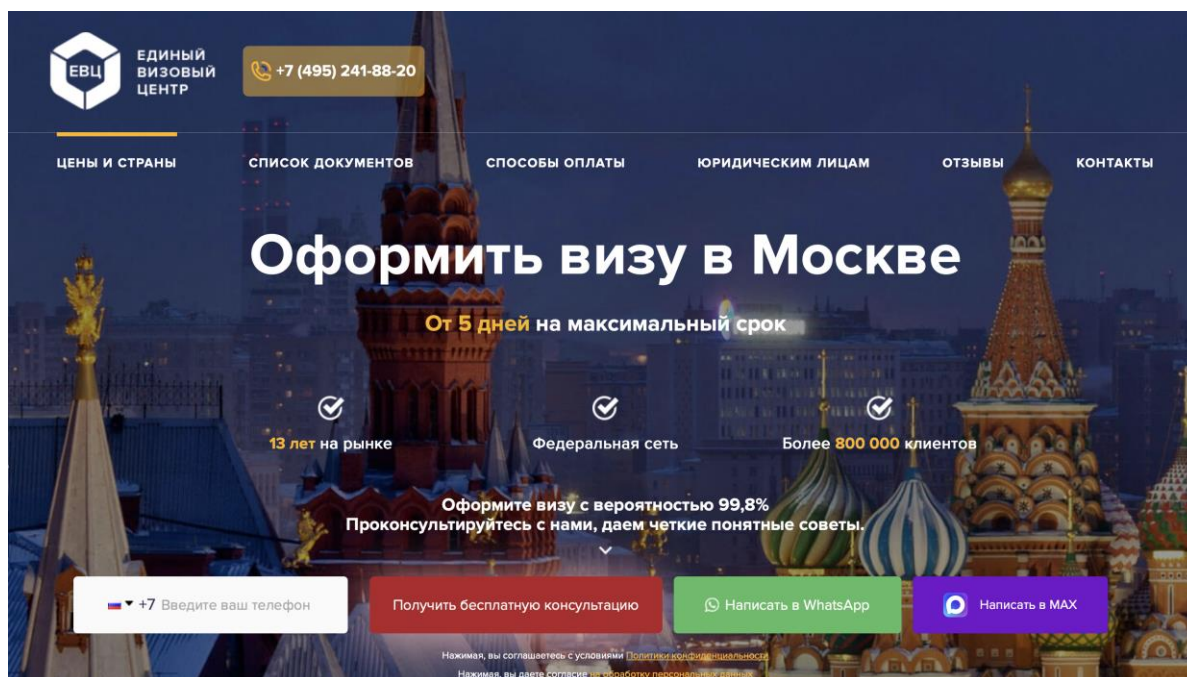


Рисунок 2 - Вид сайта единого визового центра

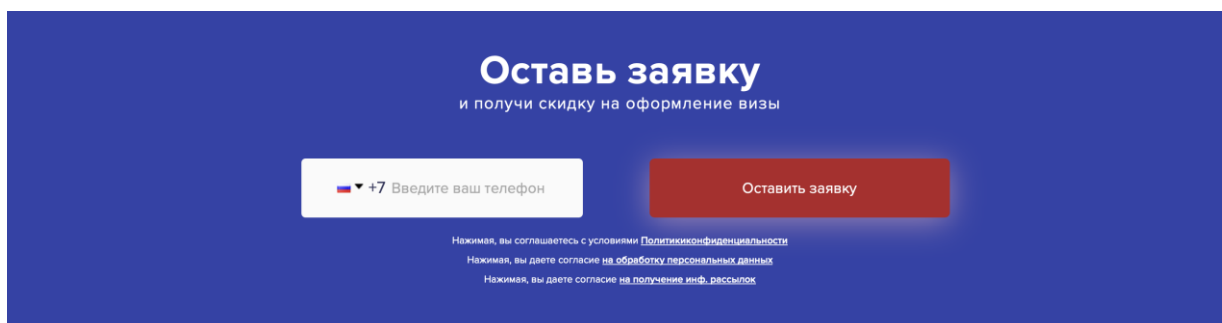


Рисунок 3 - Вид сайта единого визового центра

При разработке дизайна калькулятора из макета визового центра были заимствованы цветовая гамма (синий, красный, белый и серый цвета), стиль кнопок со скругленными углами и эффектами при наведении, шрифт. Реализация представлена на Рисунок 4

```
.calc-btn {
  width: 74px;
  height: 55px;
  border-radius: 8px;
  border: 1px solid #dddddd;
  background: #fcfcfc;
  color: #333333;
  font-size: 19px;
  cursor: pointer;
}
```

Рисунок 4 - Стиль кнопок

Для создания визуальной связи с официальным сайтом единого визового центра был создан хедер, выполненный в синем цвете (#3242AA). Это решение обеспечивает узнаваемость интерфейса и придает проекту завершённый вид.

Реализация представлена на Рисунок 5

```
.main-header {
  width: 100%;
  background-color: #3242AA;
  color: white;
  padding: 15px 0;
  display: flex;
  justify-content: center;
}
```

Рисунок 5 - Реализация хедера

Среди дополнительных элементов присутствуют кнопка перехода на GitHub, отдельный блок с подробной информацией об авторе, скрытый по умолчанию с возможностью разворачивания через элемент <details>.

ЛАБОРАТОРНАЯ РАБОТА №2 (Калькулятор HTML/CSS/JS)

- **Задача:** Создание калькулятора. Функции на JavaScript.

- Описание проекта

Проект представляет собой веб-калькулятор, реализованный с использованием HTML, CSS и JavaScript (Рисунок 6). В приложении предусмотрены стандартные арифметические операции, однако, помимо базовых вычислений, реализована авторская тематическая функция «Перевести» (calculateVisaEquivalent), специфичная для предметной области «Визовый центр». Данная функция предназначена для мгновенной конвертации вводимых сумм в эквиваленты различных мировых валют.

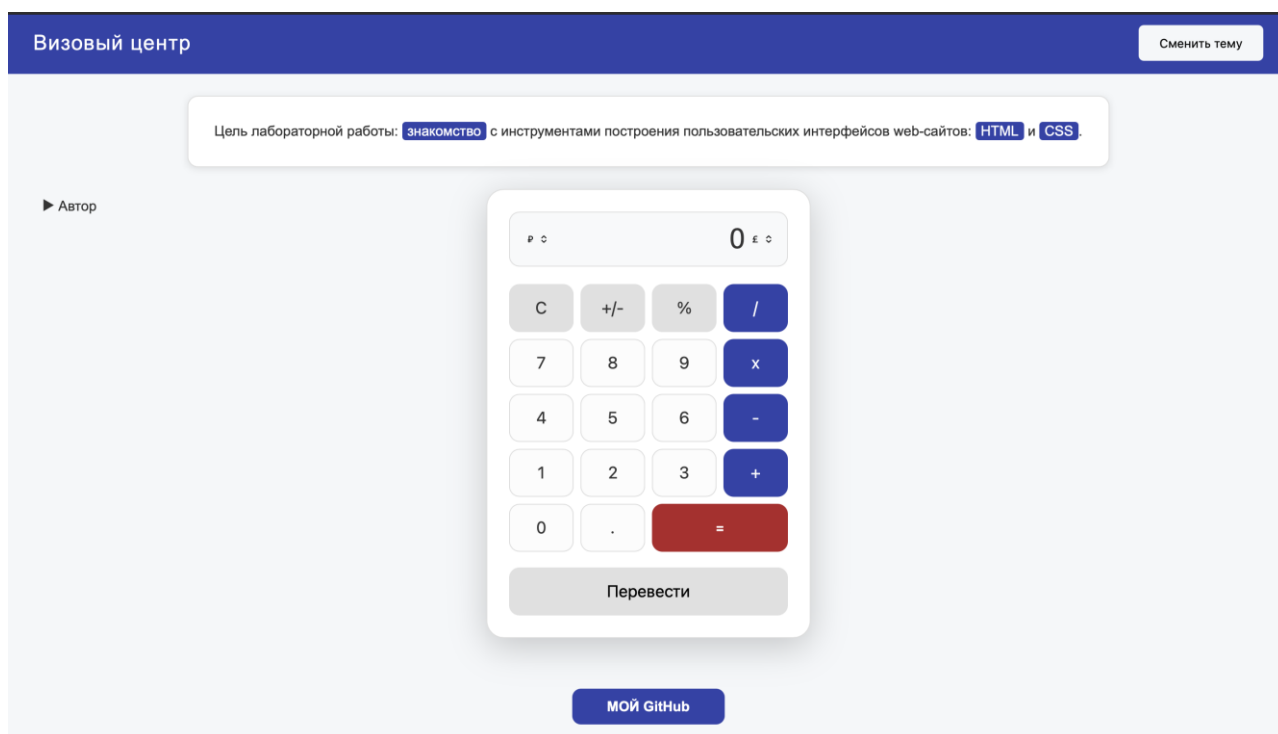


Рисунок 6 - Общий вид калькулятора

Глобальные переменные и инициализация (Рисунок 7)

Для управления состоянием приложения используются переменные, хранящие текущий ввод, предыдущее значение и выбранную операцию. Это позволяет реализовывать сложные цепочки вычислений и корректно отображать данные на дисплее калькулятора.

```
const display = document.getElementById('visa-display');
const fromCurr = document.getElementById('from-currency');
const toCurr = document.getElementById('to-currency');

let currentInput = "";
let previousValue = 0;
let operator = null;
```

Рисунок 7 - Глобальные переменные и инициализация

Реализация арифметических функций (Рисунок 8)

Каждая операция вынесена в отдельную стрелочную функцию. Названия функций адаптированы под предметную область (визовый центр): например, `addApplications` для сложения заявок или `multiplyIssuedVisas` для расчета общего количества выданных виз.

```
const addApplications = (a, b) => a + b;
const subtractDenials = (a, b) => a - b;
const multiplyIssuedVisas = (a, b) => a * b;
const divideByConsulates = (a, b) => b !== 0 ? a / b : "Err";
```

Рисунок 8 - Реализация арифметических функций

Авторская функция расчета бюджета (Рисунок 9)

Особое внимание в работе уделено функции `calculateVisaEquivalent`. Она демонстрирует работу с бизнес-логикой: мгновенную конвертацию введенной суммы в эквиваленты иностранных валют (USD, EUR, GBP). Это необходимо для быстрой оценки стоимости консульских сборов и финансовых гарантий в разных валютных единицах.

```
const calculateVisaEquivalent = () => {
  let amount = parseFloat(currentInput || display.innerText.replace(/[^d.-]/g, ''));
  if (isNaN(amount)) return;
  const result = (amount * visaExchangeRates[fromCurr.value]) / visaExchangeRates[toCurr.value];
  display.innerText = result.toFixed(2) + " " + visaCurrencySymbols[toCurr.value];
  currentInput = result.toString();
  shouldResetScreen = true;
};
```

Рисунок 9 - реализация функции `calculateVisaEquivalent`

Динамическая смена тем оформления (рисунок 9)

Для реализации кастомизации интерфейса разработан механизм переключения тем через манипуляцию классами или стилями элементов. При смене темы цвет фона страницы.

```
const themeToggle = document.getElementById('theme-toggle');

themeToggle.addEventListener('click', () => {
  document.body.classList.toggle('dark-theme');
  if (document.body.classList.contains('dark-theme')) {
    themeToggle.innerText = 'Светлая тема';
  } else {
    themeToggle.innerText = 'Сменить тему';
  }
});
```

Рисунок 10 - Реализация смены темы

ЛАБОРАТОРНАЯ РАБОТА №3. Простое веб-приложение. Верстка

- **Задача:** Знакомство с node, npm. Верстка интерфейса с карточками (страница списка с фильтрацией и страница подробнее), данные получать через mock объекты (коллекция). Добавить кнопку добавления (копировать первую карточку), кнопку удаления карточки. В хедере на обеих страницах должна быть кнопка Домой.

- **Описание проекта:** проект представляет собой двухстраничное веб-приложение, разворачиваемое с помощью npm https-server представляющее собой стилизованную по сайту единого визового центра [1] группу кнопок в виде карточек с подписями названий вид виз с возможностью поиска нужной услуги, добавления новой карточки (копирования первой) и удаления. Структура проекта представлена на Рисунок 11

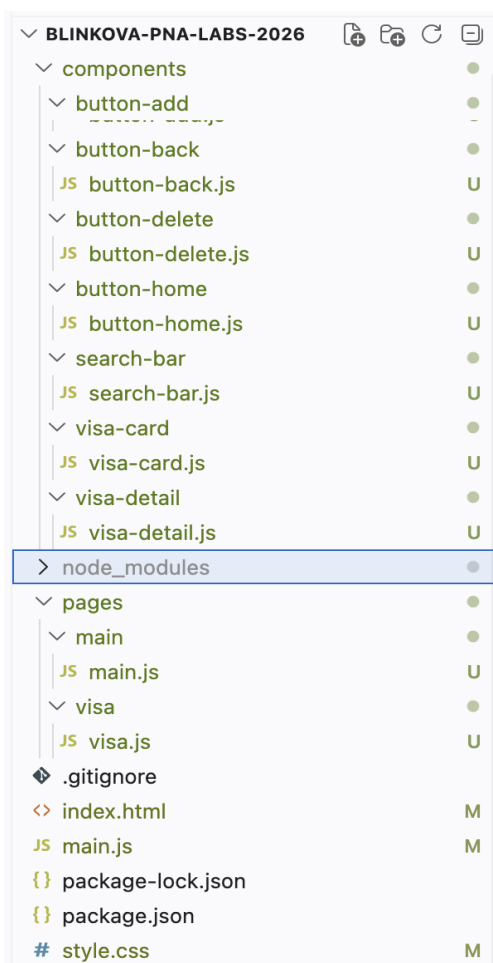


Рисунок 11 - Структура проекта

На главной странице (Рисунок 12) расположены карточки, поиск по названию карточки и кнопки для добавления и удаления карточки. Тема карточек — ВИДЫ ВИЗ

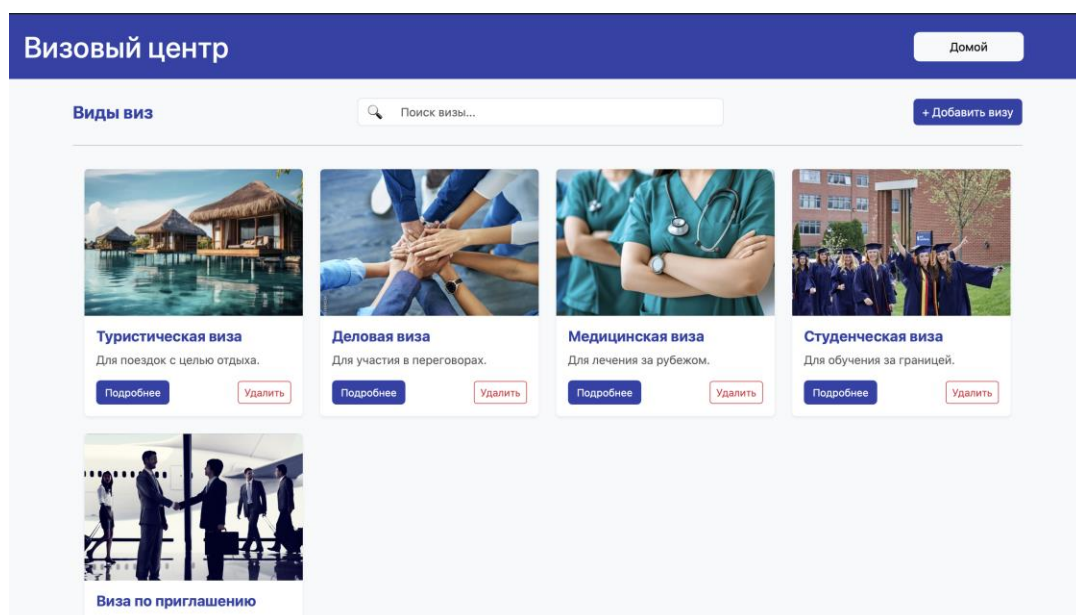


Рисунок 12 - Главная страница

При нажатии на кнопку подробнее переносит на страницу карточки (Рисунок 13) с окном, в котором содержится подробная информация о выбранном виде визы, также инструкция, кнопка “Домой” и кнопка “Назад”.

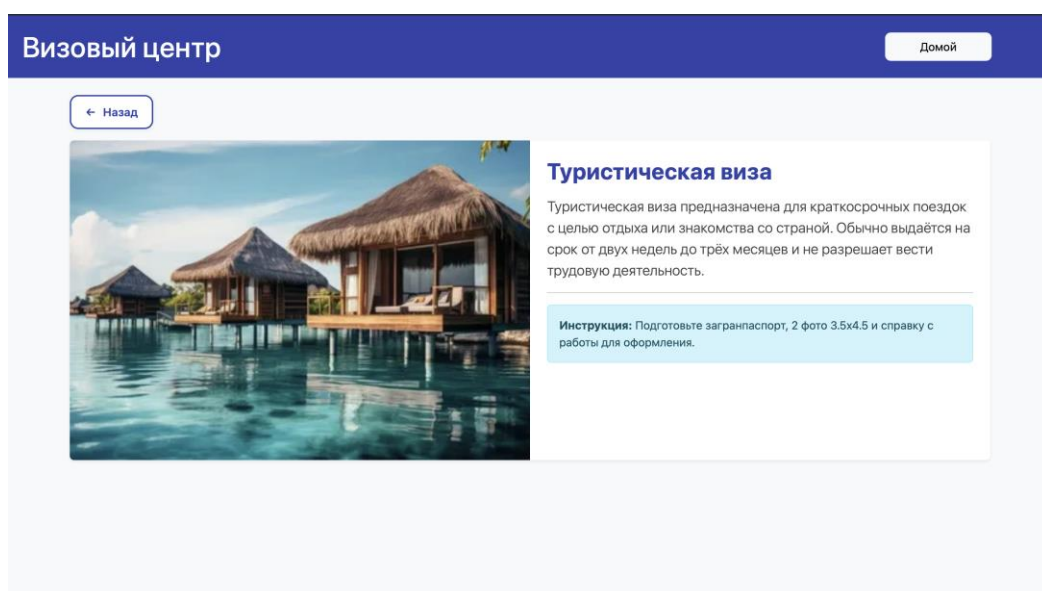


Рисунок 13 - Страница подробнее

Для обеспечения навигации по приложению была разработана кнопка возврата на главную страницу (кнопка “Домой”). Данный элемент является критически важным, так как позволяет пользователю мгновенно сбросить состояние фильтров и вернуться к полному списку виз из любой точки приложения. Реализация приведена на Рисунок 14.

```
const headerAction = document.getElementById('home-button-container');  
  
const homeBtn = new ButtonHome(headerAction);  
homeBtn.render(() => window.renderPage(MainPage));  
  
window.renderPage = (PageClass, ...args) => {  
  root.innerHTML = '';  
  const page = new PageClass(root, ...args);  
  page.render();  
};
```

Рисунок 14 - Реализация программного механизма навигации и функции возврата на главную страницу

Взаимодействие с данными организовано на уровне класса MainPage. Модель данных представлена массивом объектов visaData, где каждый объект содержит уникальный идентификатор, путь к изображению, заголовок и краткое описание услуги. Для манипуляции состоянием интерфейса реализованы следующие механизмы: поиск осуществляется по заголовкам виз, реализован метод addVisa, который создает копию существующего объекта и метод deleteVisa, который производит фильтрацию массива, исключая объект, который выбрал пользователь. Реализация приведена на Рисунок 15.

```

handleSearch(term) {
  const filtered = this.visaData.filter(v =>
    v.title.toLowerCase().includes(term.toLowerCase())
  );
  this.renderCards(filtered);
}

addVisa() {
  const newVisa = {
    ...this.visaData[0],
    id: Date.now(),
    title: `${this.visaData[0].title} (копия)`
  };
  this.visaData.push(newVisa);
  this.renderCards(this.visaData);
}

deleteVisa(id) {
  // Перезаписываем массив, исключая элемент с указанным id
  this.visaData = this.visaData.filter(v => v.id !== id);
  this.renderCards(this.visaData);
}

```

Рисунок 15 - Программная реализация методов фильтрации, добавления и удаления объектов в коллекции данных.

Таким образом, вся логика выполняется за пределами файла главной страницы, и на ней остаются только вызовы соответствующих функций для работы с кнопками, как представлено на рисунке 16.

```

render() {
  this.parent.innerHTML = this.getHTML();
  new SearchBar(document.getElementById('search-placeholder'), (term) => {
    const filtered = this.visaData.filter(v => v.title.toLowerCase().includes(term.toLowerCase()));
    this.renderCards(filtered);
  }).render();
  new ButtonAdd(document.getElementById('add-btn-placeholder')).render(() => this.addVisa());
  this.renderCards(this.visaData);
}

```

Рисунок 16 - Обработчики кнопок на главной странице

ДОМАШНЕЕ ЗАДАНИЕ

- **Задача:** Работа с коллекциями, функциями, классами.

Также на страницы сайта из лабораторной работы №3 были нативно интегрированы задачи в 2 частях из домашней работы в соответствии с вариантом, а также добавлена возможность просмотра 3D-модели оборудования с использованием библиотеки Three.js.

Для подключения модулей библиотеки three и GLTFLoader без использования сборщика (webpack/vite) в корневой файл index.html был добавлен механизм `<script type="importmap">` (Рисунок 17).

```
<script type="importmap">
  {
    "imports": {
      "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
      "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
    }
  }
</script>
```

Рисунок 17 - Подключение модулей через importmap

На страницу детального просмотра информации о визе добавлен блок рендеринга 3D-модели. Метод `init` инициализирует сцену, настраивает перспективную камеру, освещение и загружает модель в формате `.glb`, автоматически центрируя ее в контейнере (Рисунок 18).

```

// Метод инициализации 3D-сцены в классе PlanetComponent
init() {
  this.scene = new THREE.Scene();
  this.camera = new THREE.PerspectiveCamera(45, this.parent.clientWidth / this.parent.clientHeight, 0.1, 1000);
  this.camera.position.set(2, 1.5, 3);

  this.renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
  this.renderer.setSize(this.parent.clientWidth, this.parent.clientHeight);
  this.parent.appendChild(this.renderer.domElement);

  // Подключение управления мышью
  this.controls = new OrbitControls(this.camera, this.renderer.domElement);
  this.controls.enableDamping = true;

  // Настройка освещения
  this.scene.add(new THREE.AmbientLight(0xffffff, 1));
  const dirLight = new THREE.DirectionalLight(0xffffff, 2);
  dirLight.position.set(5, 5, 5);
  this.scene.add(dirLight);

  // Загрузка модели в формате .glb
  const loader = new GLTFLoader();
  loader.load('../models/planet.glb', (gltf) => {
    this.model = gltf.scene;
    this.scene.add(this.model);
    this.animate();
  });
}

```

Рисунок 18 - Инициализация сцены и загрузка 3D-модели

На Рисунок 19 представлена страница с подробной информации после рендеринга 3D-модели. У модели можно изменять масштаб, а также с помощью кнопок можно изменять положение модели.

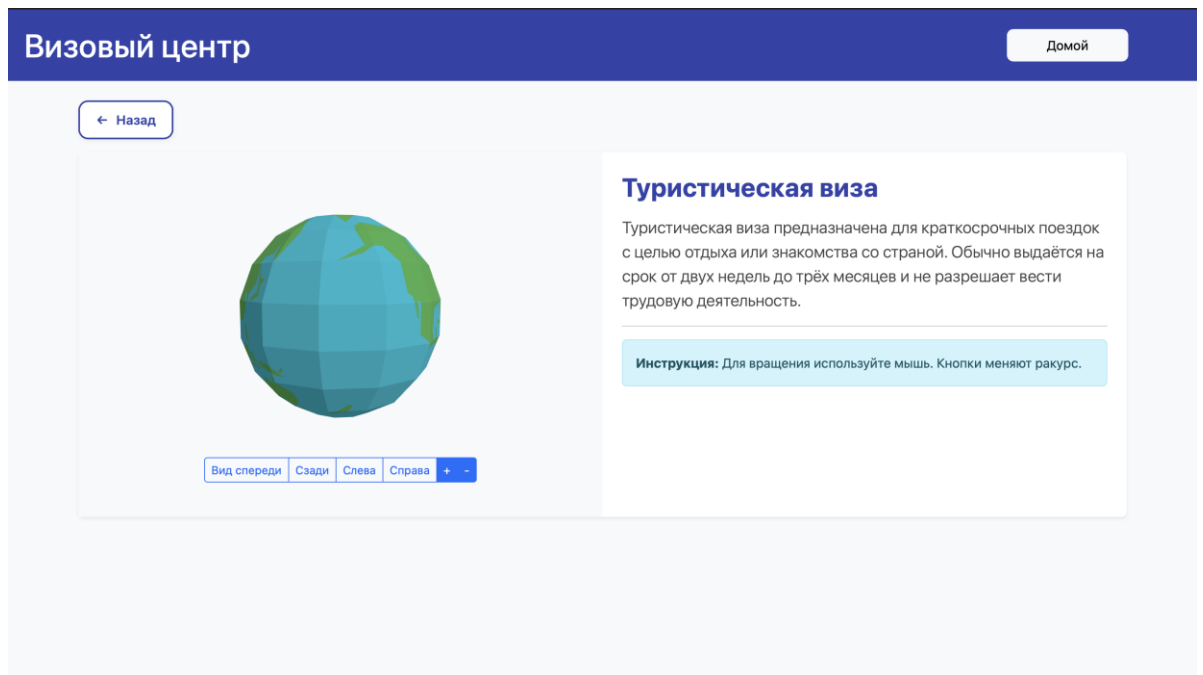


Рисунок 19 - Страница подробнее

Задание 2.3. Дана строка - последовательность, которая может состоять как из 0, так и 1. Необходимо найти максимальную последовательность 1 и вывести ее длину. Реализация приведена на Рисунок 20

```
const str = '1000000111100011111010111101111111';

const maxOnes = Math.max(...str.split('0').map(group => group.length));

console.log(maxOnes);
```

Рисунок 20 - Реализация 2.3

В соответствии с заданием 2.3, в проект интегрирован алгоритм анализа бинарных последовательностей. В контексте информационной системы визового центра данный функционал применяется для вычисления максимальной непрерывной серии одобренных заявок (где «1» — одобрение, «0» — отказ/ожидание). Применение на Рисунок 21.

```
getMaxApprovalSeries(binaryStr) {
  if (!binaryStr) return 0;
  return Math.max(...binaryStr.split('0').map(group => group.length));
}

render(data, onDetails, onDelete) {
  // 1. Отрисовка шаблона
  this.parent.insertAdjacentHTML('beforeend', this.getHTML(data));

  const cardElement = document.getElementById(`visa-card-${data.id}`);

  if (cardElement) {
    // 2. Логика кнопки "Подробнее"
    cardElement.querySelector('.details-btn').onclick = onDetails;

    // 3. ЛОГИКА ЗАДАНИЯ 2.3 (ОТВЕТ НА КАРТОЧКУ)
    cardElement.querySelector('.stats-btn').onclick = () => {
      // Берем уникальный код из данных (например, "111011")
      const sequence = data.approvalHistory || "0";

      const maxSeries = this.getMaxApprovalSeries(sequence);
      const totalCount = sequence.length;

      const output = document.getElementById(`stats-output-${data.id}`);

      if (output) {
        output.textContent = `Пик: ${maxSeries} из ${totalCount} одобрений`;
        output.style.display = 'inline-block';
      }
    }
  }
}
```

Рисунок 21 - Применение 2.3 в лабораторной №3

Задача 3.5: Напишите функцию merge, которая будет принимать на вход несколько объектов (любое количество), и возвращать единственный объект, содержащий все поля из всех объектов. Если одно и то же поле было в нескольких объектах, необходимо оставить значение, которое встретилось раньше. Реализация задачи представлена на **Error! Reference source not found.**

```
merge(...objects) {  
  const result = {};  
  let i = 0;  
  
  if (objects.length === 0) return result;  
  
  do {  
    const currentObj = objects[i];  
    // Перебор ключей текущего объекта  
    for (const key in currentObj) {  
      // Проверка: берем только собственные свойства и только если ключа еще нет в результате  
      if (currentObj.hasOwnProperty(key) && !(key in result)) {  
        result[key] = currentObj[key];  
      }  
    }  
    i++;  
  } while (i < objects.length);  
  
  return result;  
}
```

Рисунок 22 - Реализация задачи 3.1

В информационной системе визового центра данный алгоритм используется для динамического формирования перечня **необходимых документов** для каждой конкретной категории виз.

Логика работы в интерфейсе:

1. Имеется объект generalDocs с базовым набором документов, общим для всех (паспорт, фото).
2. Для каждой карточки в базе данных определен объект extraDocs с уникальными требованиями (например, «Бронь отеля» или «Справка о доходах»).
3. При нажатии пользователем на кнопку «**Проверить документы**», компонент VisaCardComponent вызывает функцию merge, объединяя эти объекты.

Применение показано на Рисунок 23.


```

cardElement.querySelector('.merge-btn').onclick = () => {
  const generalDocs = { passport: "Загранпаспорт", photo: "Фото" };
  const countryDocs = data.extraDocs || { insurance: "Страховка" };

  // Слияние объектов
  const fullPackage = this.merge(generalDocs, countryDocs);

  const output = document.getElementById(`merge-output-${data.id}`);
  output.textContent = "Нужно: " + Object.values(fullPackage).join(", ");
};

```

Рисунок 23 - Использование функции из 3.1 в лабораторной №3

Благодаря использованию merge, система гарантирует отсутствие дубликатов в списке документов (например, если «Фото» указано в обоих наборах, оно отобразится один раз) и позволяет гибко дополнять требования в зависимости от выбранной страны.

ЛАБОРАТОРНАЯ РАБОТА №4. Реализация собственного API на Node.js

Задача: Реализация собственного API на Node.js. Тестирование через Postman 5 методов: получение списка (в том числе с фильтрацией), получение одной записи, добавление, редактирование, удаление.

Описание проекта

В рамках лабораторной работы было разработано серверное приложение на базе Node.js и микрофреймворка Express.js (использовался JavaScript). Приложение предоставляет полноценный REST API для управления каталогом услуг (карточками). Для персистентности данных используется локальный JSON-файл (stocks.json), который эмулирует работу базы данных.

Архитектура и реализация

Проект имеет слоистую архитектуру, разделяющую логику маршрутизации, обработки запросов и работы с данными: - Routes (stocks.js) – определяет конечные точки и связывает их с контроллерами. - Controllers (stocksController.js) – отвечает за валидацию входящих данных (например, проверку корректности id или наличия обязательных полей src, title, text) и формирование HTTP-ответов. - Services (stocksService.js, fileService.js) – содержит основную бизнес-логику CRUD-операций и методы для чтения/записи массива объектов в файл с помощью модуля fs. - Middleware – реализованы промежуточные обработчики для логирования каждого запроса в консоль и глобального перехвата ошибок.

Тестирование API в Postman

Тестирование работоспособности проводилось с помощью инструмента Postman. Сервер обрабатывает запросы по базовому адресу <http://localhost:3005/stocks>.

1. **Получение списка всех записей (GET)** Выполнен запрос на получение массива всех доступных карточек. (Рисунок 24)

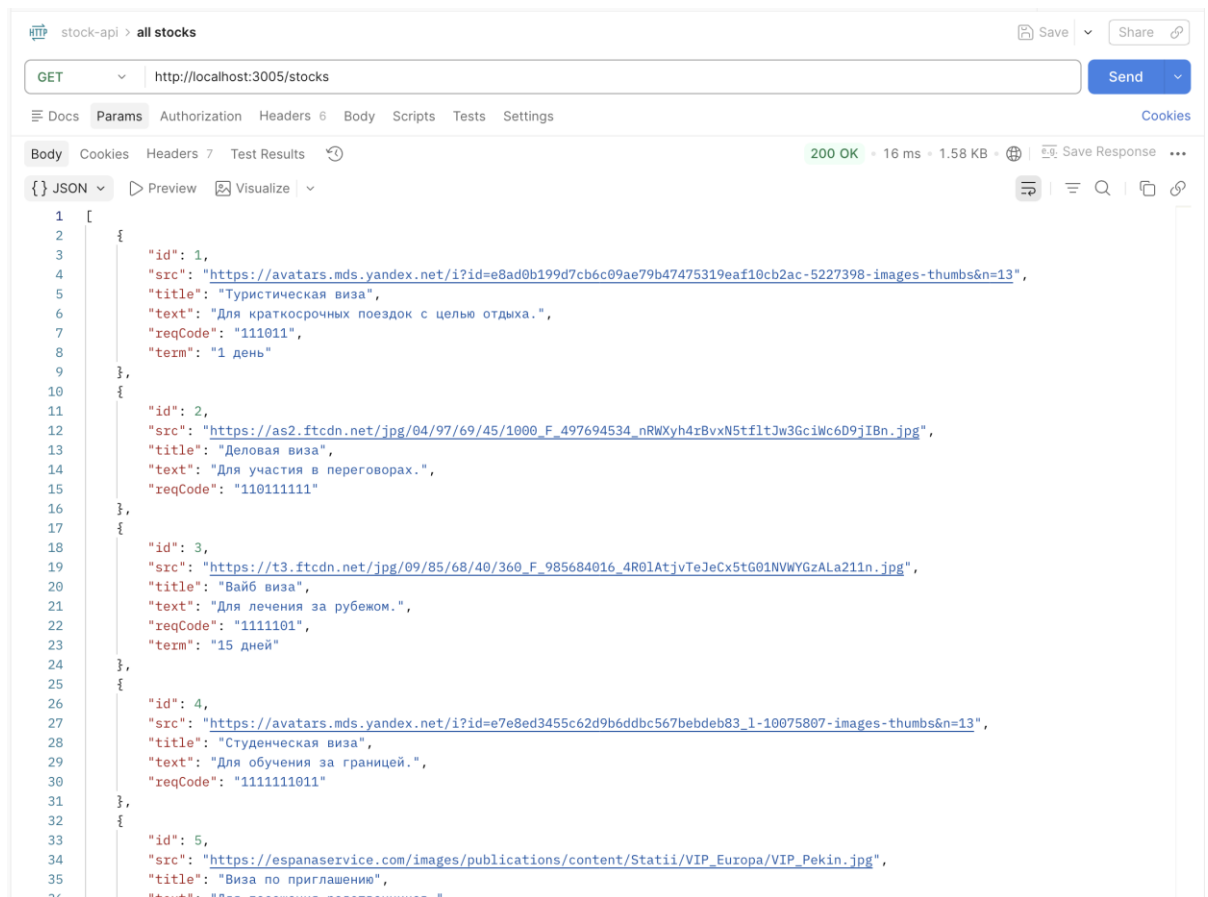


Рисунок 24 - Запрос GET для получения всех записей

2. **Создание новой записи (POST)** В теле запроса (Body -> JSON) переданы поля src, title и text. Сервер автоматически сгенерировал новый id и вернул созданный объект со статусом 201 Created. (Рисунок 25)

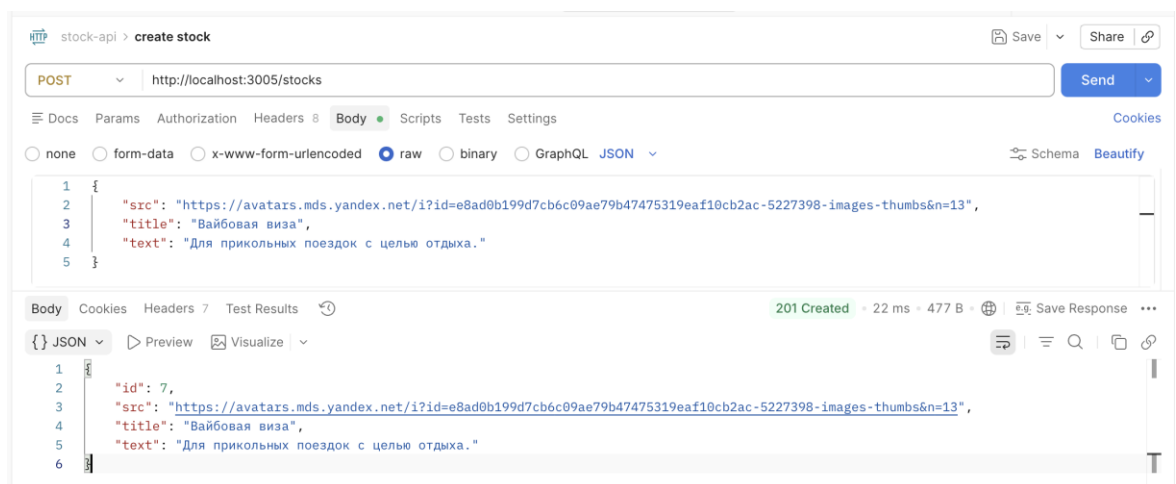


Рисунок 25 - Запрос POST для создания карточки

3. **Получение записи по ID (GET)** Выполнен запрос с указанием конкретного идентификатора в параметрах пути (например, /stocks/1). (Рисунок 26)

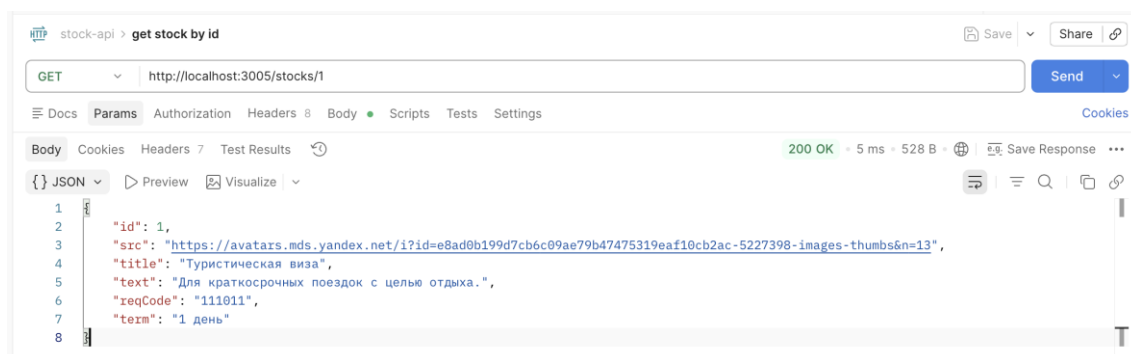


Рисунок 26 - Запрос GET для получения карточки по id

4. **Редактирование записи (PATCH)** Протестировано частичное обновление данных. В теле запроса переданы только те поля, которые необходимо изменить (например, обновлен только title). (Рисунок 27)

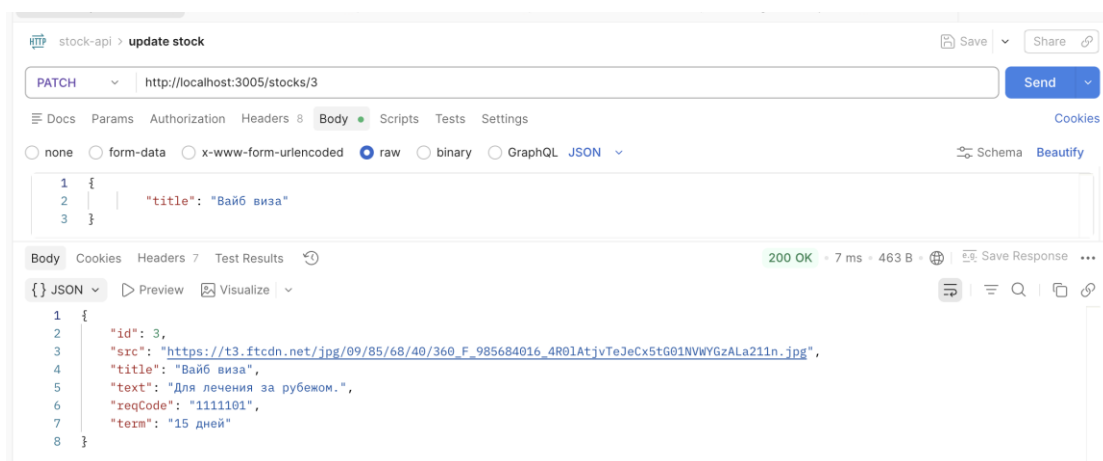


Рисунок 27 - Запрос PATCH для обновления карточки

5. **Фильтрация записей (GET с параметром)** Выполнен GET-запрос с использованием query-параметра ?title=... для проверки логики фильтрации на стороне сервиса. (Рисунок 28)

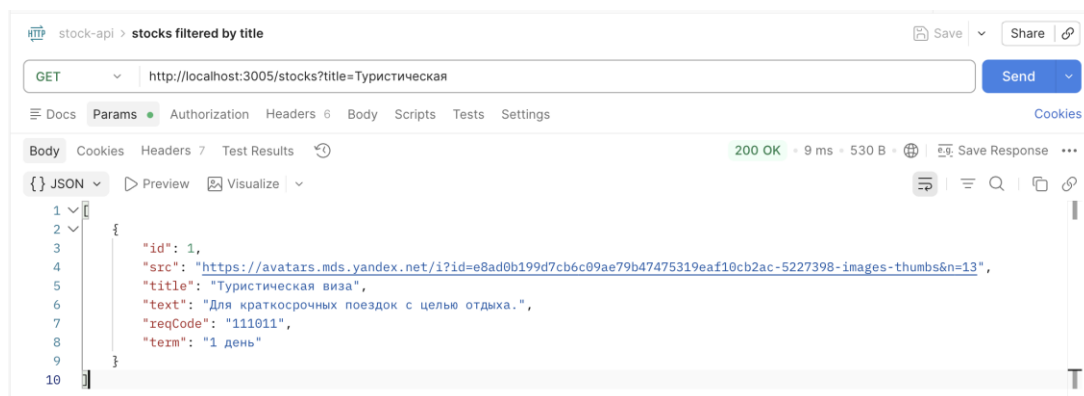


Рисунок 28 - Запрос GET с фильтрацией по title

6. **Удаление записи (DELETE)** Выполнен запрос на удаление существующей карточки по её ID. В случае успешного удаления сервер возвращает статус 204 No Content.(Рисунок 29)

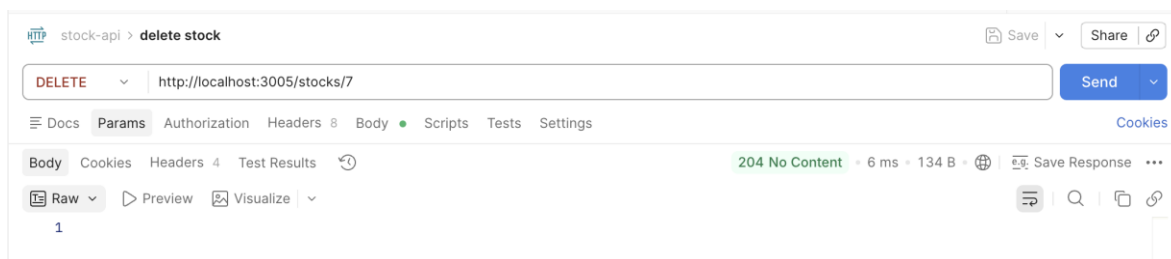


Рисунок 29 - Запрос DELETE для удаления карточки

ЗАКЛЮЧЕНИЕ

- В рамках первой лабораторной работы был реализован стилизованный статический веб-калькулятор, позволивший освоить основы HTML и CSS.

- Во второй лабораторной работе калькулятору была добавлена интерактивность с помощью Javascript, включая математические функции и динамическую смену тем.

- В третьей лабораторной работе было разработано двухстраничное веб-приложение-галерея, реализованное с помощью Bootstrap.

- В домашнем задании были разработаны и включены в общий проект несколько алгоритмов обработки данных на Javascript

- В четвёртой лабораторной работе был разработан REST API с помощью Node.js и RestJS с поддержкой CRUD, а также протестирован с помощью Postman.

ЛИТЕРАТУРА

1.Сайт единый визовый центр – официальная домашняя страница [Электронный ресурс] – URL: <https://единый-визовый-центр.рф/msk/>.

2.Лекции по курсу XML-технологии и методические указания по выполнению лабораторных работ [Электронный ресурс]. – URL: <https://github.com/iu5git/JavaScript/blob/main>